

苏州大学实验报告

院系	计算机学院	年级专业	21 计科	姓名	方浩楠	学号	2127405048
课程名称	操作系统课程实践					成绩	
指导教师	王红玲	同组实验者	无	实验日期	2024. 5. 22		

实 验 名 称 实验九 字符设备驱动程序编写

一. 实验目的

- (1) 了解 Linux 字符设备管理机制。
- (2) 学习字符设备的基本管理方法。
- (3) 学会编写简单的字符设备驱动程序的方法。

二. 实验内容

编写一个字符设备驱动程序，要求能对该字符设备执行打开、读、写、I/O 控制和关闭这 5 项基本操作。并编写测试程序，测试添加的字符设备的正确性。

三. 实验步骤

1. 编写字符设备驱动程序

(1) 编写设备驱动源程序，即编写内核模块文件 chardev.c 和 Makefile 文件。对字符设备的实现为：同一时间只允许被一个进程打开一次，每次打开会将内容重置为一个字符串：I already told you %d times Hello world，其中%的为打开的次数。对字符设备的读取默认会输出这个字符串，写入则会覆盖该字符串。

chardev.c:

```
#include<linux/init.h>
#include<linux/kernel.h>
#include<linux/slab.h>
#include<linux/module.h>
#include<linux/moduleparam.h>
#include<linux/fs.h>
#include<linux/uaccess.h>
#define SUCCESS 0
#define DEVICE_NAME "chardev"
#define BUF_LEN 80
static int Major;
static int Device_Open =0;
static char msg[BUF_LEN];
static char *msg_Ptr;
```

```

static int device_open(struct inode *inode, struct file *file);
static int device_release(struct inode *inode, struct file *file);
static ssize_t device_read(struct file *filp, char *buffer, size_t length, loff_t *offset);
static ssize_t device_write(struct file *filp, const char *buff, size_t length, loff_t *off);
static struct file_operations fops = {
    .read = device_read,
    .write = device_write,
    .open = device_open,
    .release = device_release
};

//打开设备

static int device_open(struct inode *inode, struct file *file)
{
    static int counter = 0;
    if(Device_Open)
        return -EBUSY;
    Device_Open++;
    sprintf(msg, "I already told you %d times Hello world\n", counter++);
    msg_Ptr = msg;
    try_module_get(THIS_MODULE);
    return SUCCESS;
}

//释放设备

static int device_release(struct inode *inode, struct file *file)
{
    Device_Open--;
    module_put(THIS_MODULE);
    return 0;
}

//读设备

static ssize_t device_read(struct file *filp, char *buffer, size_t length, loff_t *offset)
{
    if(*msg_Ptr == 0)
        return 0;
    copy_to_user(buffer, msg_Ptr, length);
    return strlen(msg);
}

//写设备

static ssize_t device_write(struct file *filp, const char *buff, size_t length, loff_t *off)
{

```

```

        printk("<1> Sorry this operation isn't supported\n "); //未实现 write 功能

        return -EINVAL;
    }

//初始化字符设备

int init_chardev_module(void)
{
    Major = register_chrdev(0,DEVICE_NAME,&fops);
    if(Major <0){
        printk("Registering the character device failed with %d \n ", Major);
        return Major;
    }
    printk("<1> I was assigned major number %d ", Major);
    printk("<1> the drive, create a dev file");
    printk("<1> mknod /dev/hello c %d 0.\n" ,Major);
    printk("<1> I was assigned major number %d ", Major);
    printk("<1> the device file\n" );
    printk("<1> Remove the file device and module when done\n");
    return 0;
}

//关闭字符设备

void exit_chardev_module(void)
{
    unregister_chrdev(Major, DEVICE_NAME);
}

MODULE_LICENSE("Dual BSD/GPL");
module_init(init_chardev_module);
module_exit(exit_chardev_module);

```

Makefile:

```

TARGET=chardev
obj-m += $(TARGET).o
cc=gcc
KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)
all: modules
modules:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:

```

```
$(MAKE) -C $(KDIR) M=$(PWD) clean
```

(2) 使用 `make` 命令编译驱动模块。

```
make
```

(3) 使用 `insmod` 命令安装驱动模块。

```
insmod chardev.ko  
lsmod | grep chardev
```

(4) 创建字符设备文件，语法格式：`mknod /dev/文件名 c 主设备号 次设备号`

```
mknod /dev/hello c <major> 0
```

(5) 编写测试程序 `test.c`，访问创建的字符设备文件，测试其打开、关闭、读取和写入。

```
#include<stdio.h>  
#include<stdlib.h>  
#include<unistd.h>  
#include<sys/stat.h>  
#include<fcntl.h>  
#include<sys/types.h>  
int main()  
{  
    char buf[4096] = {"I have already told you 1 time hello world"};  
    int fd =open("/dev/hello",O_RDWR );  
    int ret = read(fd ,buf ,sizeof(buf));  
    buf[ret] = '\0';  
    printf("%s\n",buf);  
}
```

并使用 `gcc` 编译这个字符设备文件，然后运行。

```
# gcc test.c -o chardev-test
```

```
# ./chardev-test
```

(6) 使用 `rmmod` 卸载模块。

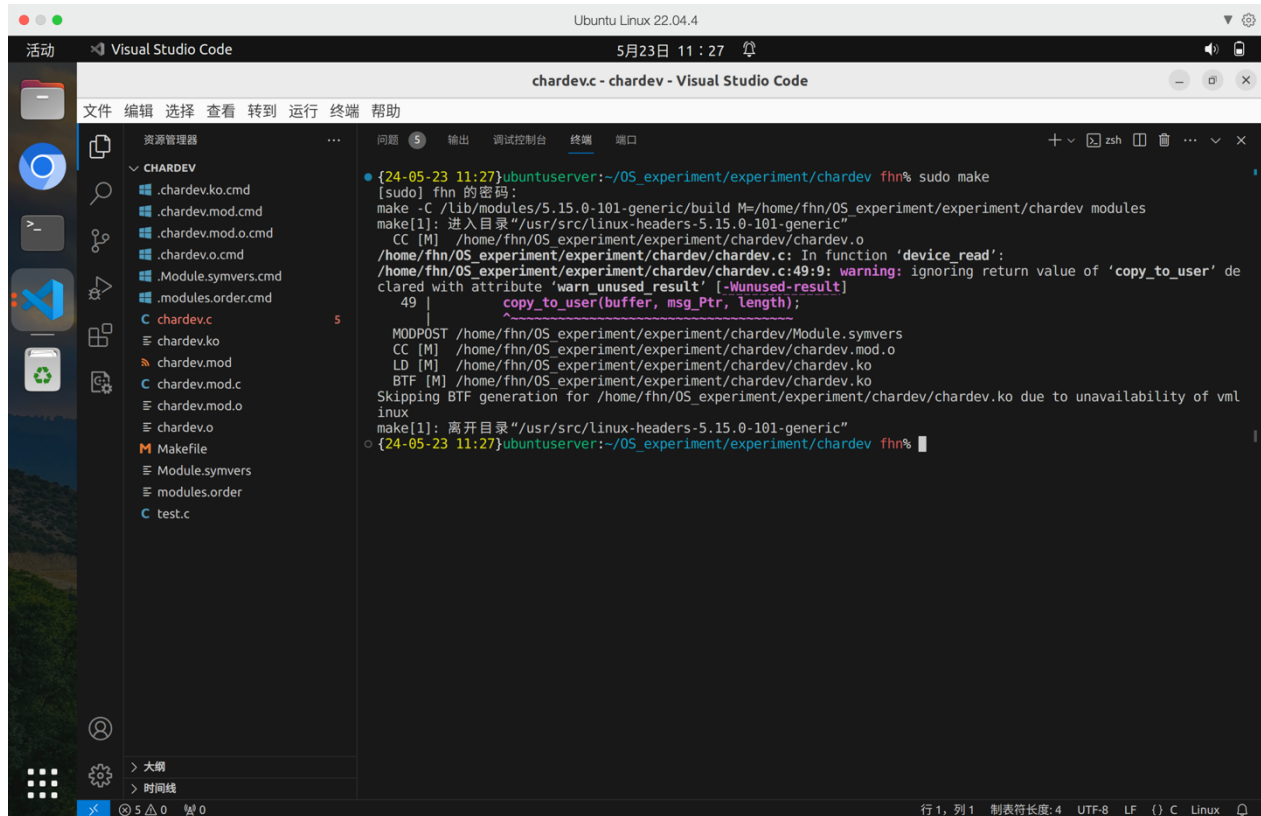
```
# rmmod chardev
```

(7) 使用 `rm` 命令删除所创建的字符设备文件。

```
# rm /dev/hello
```

四. 实验结果

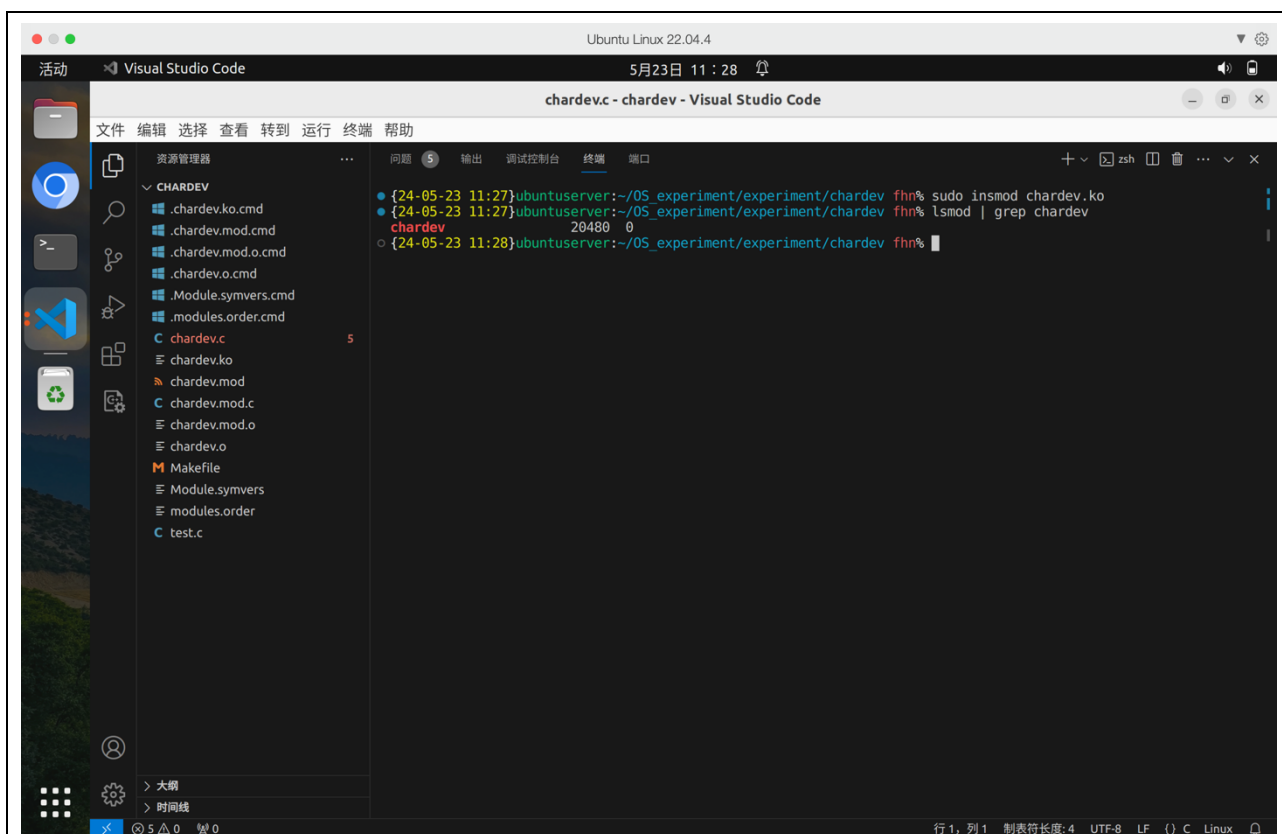
1.使用 `make` 命令编译字符设备驱动模块



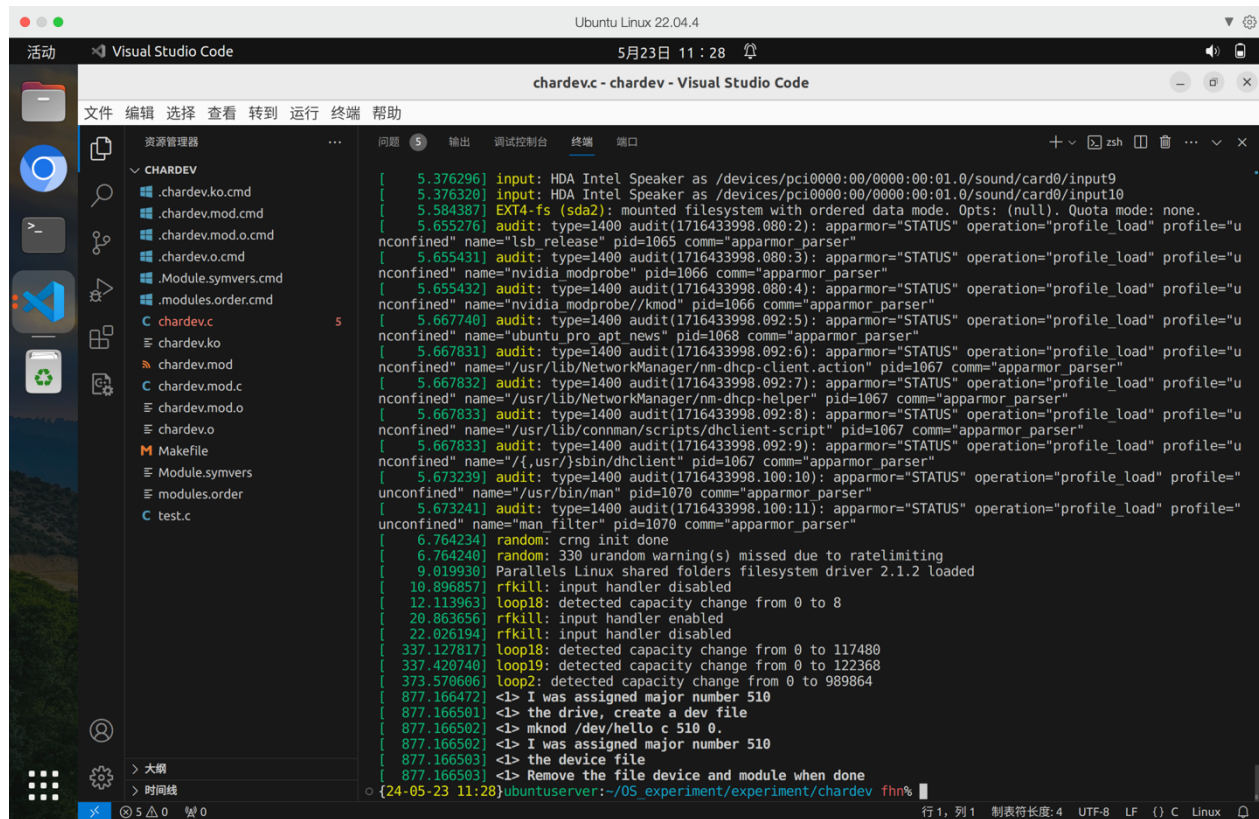
The screenshot shows the Visual Studio Code interface with a terminal window open. The terminal displays the output of the `make` command, which successfully compiles the `chardev` module. The output includes the following lines:

```
[24-05-23 11:27]ubuntuuser:~/OS_experiment/experiment/chardev fhn% sudo make
[sudo] fhn 的密码:
make -C /lib/modules/5.15.0-101-generic/build M=/home/fhn/OS_experiment/experiment/chardev modules
make[1]: 进入目录"/usr/src/linux-headers-5.15.0-101-generic"
CC [M] /home/fhn/OS_experiment/experiment/chardev/chardev.o
/home/fhn/OS_experiment/experiment/chardev/chardev.c: In function 'device_read':
/home/fhn/OS_experiment/experiment/chardev.c:49:9: warning: ignoring return value of 'copy_to_user' de
clared with attribute 'warn_unused_result' [-Wunused-result]
    49 |         copy_to_user(buffer, msg_ptr, length);
        |         ^~~~~~
MODPOST /home/fhn/OS_experiment/experiment/chardev/Module.symvers
CC [M] /home/fhn/OS_experiment/experiment/chardev/chardev.mod.o
LD [M] /home/fhn/OS_experiment/experiment/chardev/chardev.ko
BTF [M] /home/fhn/OS_experiment/experiment/chardev/chardev.ko
Skipping BTF generation for /home/fhn/OS_experiment/experiment/chardev/chardev.ko due to unavailability of vml
inux
make[1]: 离开目录"/usr/src/linux-headers-5.15.0-101-generic"
[24-05-23 11:27]ubuntuuser:~/OS_experiment/experiment/chardev fhn% 
```

2. 使用 `insmod chardev.ko` 命令安装编译好的字符驱动模块 使用 `lsmod | grep chardev` 命令查看该模块是否装载成功



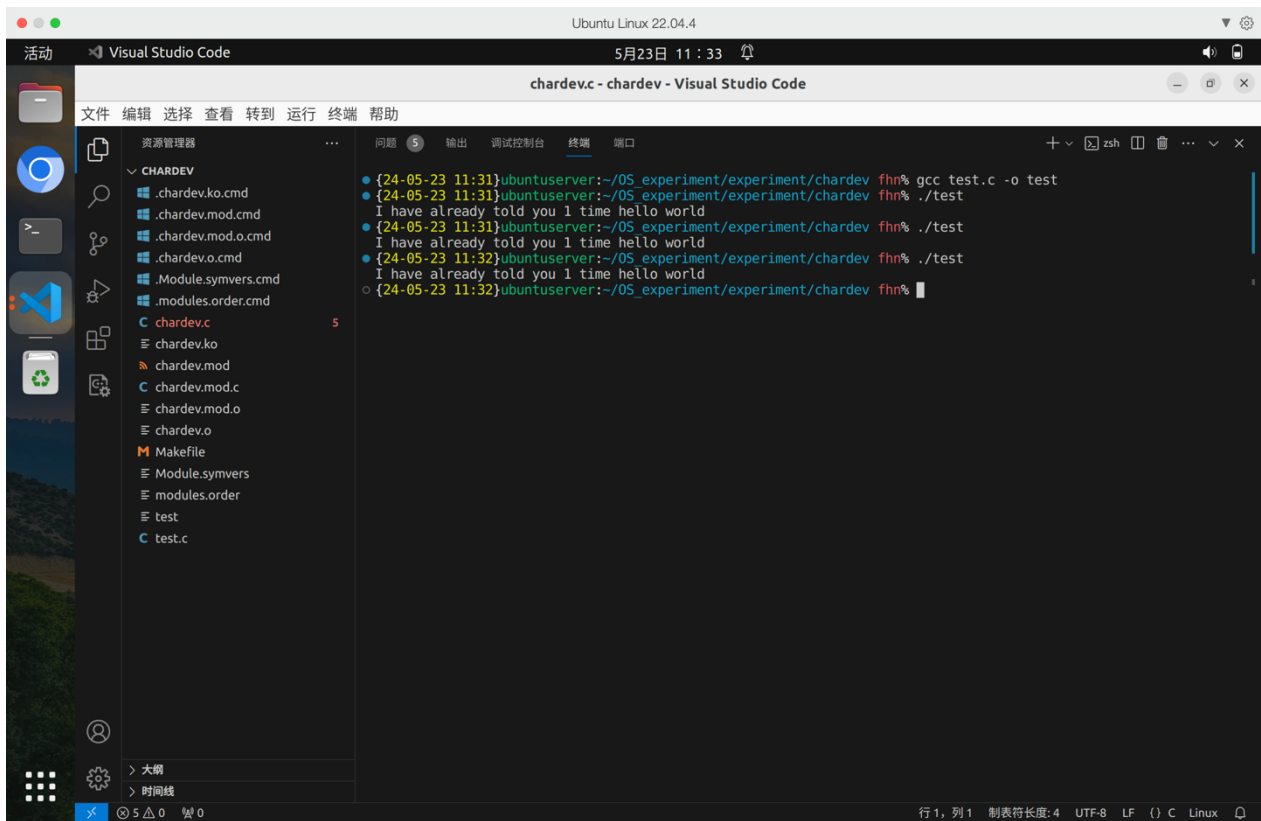
3. 使用 dmesg 命令查看系统分配的主设备号



4. 根据输出的主设备号，利用 mknod 命令创建设备

```
sudo rmmod chardev
{24-05-23 11:30}ubuntuuser:~/05_experiment/experiment/chardev fhn% sudo mknod /dev/hello c 510 0
{24-05-23 11:30}ubuntuuser:~/05_experiment/experiment/chardev fhn%
```

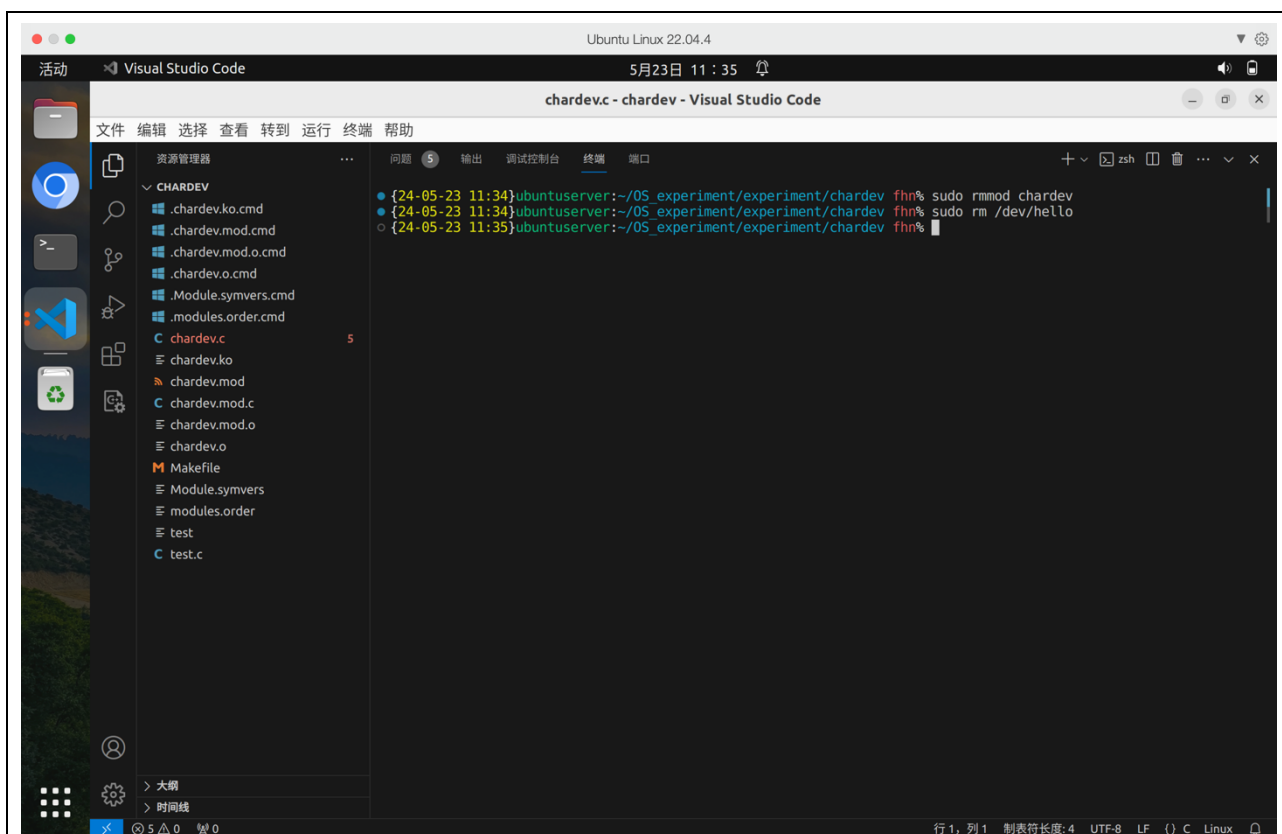
5. 编译并运行测试程序 test.c



```
Ubuntu Linux 22.04.4
5月23日 11:33
chardev.c - chardev - Visual Studio Code
文件 编辑 选择 查看 转到 运行 终端 帮助
资源管理器
. chardev.ko.cmd
. chardev.mod.cmd
. chardev.mod.o.cmd
. chardev.o.cmd
Module.symvers.cmd
modules.order.cmd
chardev.c
chardev.ko
chardev.mod
chardev.mod.c
chardev.mod.o
chardev.o
Makefile
Module.symvers
modules.order
test
test.c
问题 5 输出 调试控制台 终端 端口
{24-05-23 11:31}ubuntuuser:~/05_experiment/experiment/chardev fhn% gcc test.c -o test
{24-05-23 11:31}ubuntuuser:~/05_experiment/experiment/chardev fhn% ./test
I have already told you 1 time hello world
{24-05-23 11:31}ubuntuuser:~/05_experiment/experiment/chardev fhn% ./test
I have already told you 1 time hello world
{24-05-23 11:32}ubuntuuser:~/05_experiment/experiment/chardev fhn% ./test
I have already told you 1 time hello world
{24-05-23 11:32}ubuntuuser:~/05_experiment/experiment/chardev fhn%
```

输出结果符合预期

6. 使用 rmmod 卸载模块，并使用 rm 命令删除所创建的字符设备文件。



五. 实验总结

通过本次实验，我了解了 Linux 字符设备和块设备的管理机制，掌握了 管理方法，学会了编写简单的字符设备驱动程序和块设备驱动程序的方法。